
Neural Networks for NLP - Sheet 1

10.11.2025

Deadline: 24.11.2025 - 13:15, on OLAT as **pdf/.py/.ipynb** and/or after the lecture

Task 1: Linear- and Nonlinear Models

Assume we are given two lists - one containing 1,000 randomly selected Spanish words and another containing 1,000 randomly selected English words. Given a new word that is either English or Spanish, we want to use these lists to predict its language. We will first vectorize all words, i.e., find a sensible mapping ϕ that maps any word to some vector in $\mathbb{R}^n$.

- (a) Describe a vectorization mapping for words for this problem.
- (b) When given a new word, w , we use vectorization to obtain $\phi(w)$ and compare it to the other 2,000 vectors in order to determine its language. For this purpose, should we use a linear model (such as an SVM), or should we use a non-linear approach (such as neural networks)? Explain.
- (c) Suppose we use a neural network to obtain such a classifier. Describe the input and output of such a network as well as its training procedure using the given two lists.

Task 2: Preprocessing

In the Materials folder, you will find `tutorial-1.ipynb`. The task involves several steps of text preprocessing to evaluate changes in the performance of a text classification model, specifically in terms of accuracy.

- (a) Remove stopwords and special tokens from the text (Bag of Words model). Document how the removal of these elements impacts the accuracy of the model.
- (b) Manually apply tf-idf to the text. Do not use any external packages for this task. Report on how the accuracy of the model changes as a result of this transformation.
- (c) Determine and report the minimal sufficient vocabulary size. This should be based on the impact of vocabulary size on the model's accuracy. Provide a brief analysis of how reducing or increasing the vocabulary size affects performance.

Each part of this task requires careful manipulation of text data and assessment of outcomes, contributing to a comprehensive understanding of basic text preprocessing techniques in natural language processing.

Submission: Please submit your completed `tutorial-1.ipynb` file. Your documentation, reports, and analysis for parts (a), (b), and (c) should be included as Markdown cells within the notebook, alongside your code and any generated figures.

Task 3: Implementing a Byte Pair Encoding (BPE) Tokenizer

In this task, you will implement key components of a Byte Pair Encoding (BPE) tokenizer, a fundamental tool in natural language processing for subword tokenization. You are provided with several Python files containing skeleton code for tokenizer classes and associated helper functions. Your task is to complete specific functions within these files to build a functioning tokenizer.

1. Implement the `get_stats` function in `base.py`. This function computes the frequency of each pair of consecutive tokens in a list of token IDs. A better working of the function is given in the function definition.
2. Implement the `merge` function in `base.py`, which replaces all occurrences of a specified token pair with a new token ID in a list of token IDs.
3. Complete the `building_merges` method in the `BasicTokenizer` class in `basic.py`, utilizing the `get_stats` and `merge` functions to build the tokenizer's vocabulary. The method involves creating merges and building vocabulary from a training corpus to determine the most frequent token pairs and applying the merge operations accordingly which will be used by the encode and decode function.
4. Complete the `building_merges` method in the `RegexTokenizer` class in `regex.py`, which extends the `BasicTokenizer`. What you need to do is use the predefined `Regex` method before applying it to the `get_stats` and the `merge` (The way you did in `BasicTokenizer`, but you just need to use the `Regex` pattern before.)

Submission: Please submit the modified Python files (`base.py`, `basic.py`, and `regex.py`) containing your completed functions.